

Rapport de Projet

Développement Mobile

2 ère informatique
groupe D

Application de réservation de spectacles

Réalisé par :
Skander Hakouna
Encadré par:
Mme Olfa Lamouchi

Année universitaire :
2024/2025

TABLE DES MATIÈRES

1. Introduction Générale	04
1.1 Description du Projet	04
1.2 Objectifs	05
1.3 Technologies utilisées	06
3. Diagrammes UML	07
3.1 Diagramme de cas d'utilisation	07
3.2 Diagramme de classes	08
4. Base de Données & API de Connexion	09
4.1 Modélisation de la base de données	09
4.2 API Spring Boot : structure et endpoints	10
4.3 Sécurité et authentification utilisateur	11
4.4 Historique de réservations	11
5. Arborescence des Interfaces	12
5.1 Navigation entre les écrans	12
5.2 Arbre des interfaces utilisateur (schéma)	13
6. Outils Utilisés & Difficultés Rencontrées	14
6.1 Environnement de développement	14
6.2 Outils tiers (Postman, Git, Workbench...)	14
6.3 Problèmes rencontrés et solutions apportées	15

7. Interfaces de l'Application		16
7.1 Écran d'accueil et login		16
7.2 Liste des spectacles		17
7.3 Détails d'un spectacle		18
7.4 Réservation		19
7.5 Profil utilisateur		20
8. Annexe : Code des Parties Importantes		21
8.1 Code backend (Spring Boot)		21
8.2 Code frontend Android (activités & adaptateurs)		24
9. Conclusion Générale		26

1. Introduction Générale

1.1 Description du projet

L'application ShowTime est une plateforme mobile Android permettant aux utilisateurs de découvrir des spectacles culturels disponibles en Tunisie, d'en consulter les détails, et de réserver des billets directement depuis leur smartphone. Elle s'adresse à un public large, allant des amateurs de théâtre et musique aux familles à la recherche d'événements divertissants.

Le parcours utilisateur est pensé pour être simple et efficace. Dès l'ouverture de l'application, l'utilisateur accède à une liste de spectacles disponibles, avec la possibilité de filtrer par ville et par date. En cliquant sur un spectacle, l'utilisateur accède à sa fiche détaillée comprenant le titre, la description, la date, l'heure, le lieu, et l'affiche visuelle du spectacle. Il peut ensuite réserver un ou plusieurs billets, qu'il soit connecté ou non.

Si l'utilisateur est connecté, la réservation est automatiquement liée à son compte et il reçoit une confirmation par e-mail. S'il n'est pas connecté, il est redirigé vers une formulaire de réservation simplifiée avec nom complet et e-mail. L'application propose également un espace Profil, affichant les informations personnelles et l'historique des réservations.

Le côté backend de l'application est assuré par un serveur Spring Boot connecté à une base de données MySQL. Il permet la gestion des comptes utilisateurs, des réservations et des spectacles, et expose une API RESTful pour communiquer avec l'application Android.

1. Introduction Générale

1.2 Objectifs

Les objectifs de ce projet sont multiples :

- Mettre en pratique les compétences acquises en développement mobile Android avec Java.*
- Comprendre et implémenter une architecture frontend–backend complète.*
- Gérer la persistance des données avec une base de données relationnelle (MySQL).*
- Assurer la communication entre l'application Android et l'API Spring Boot via des appels REST.*
- Concevoir une interface utilisateur fluide, moderne et intuitive.*

1. Introduction Générale

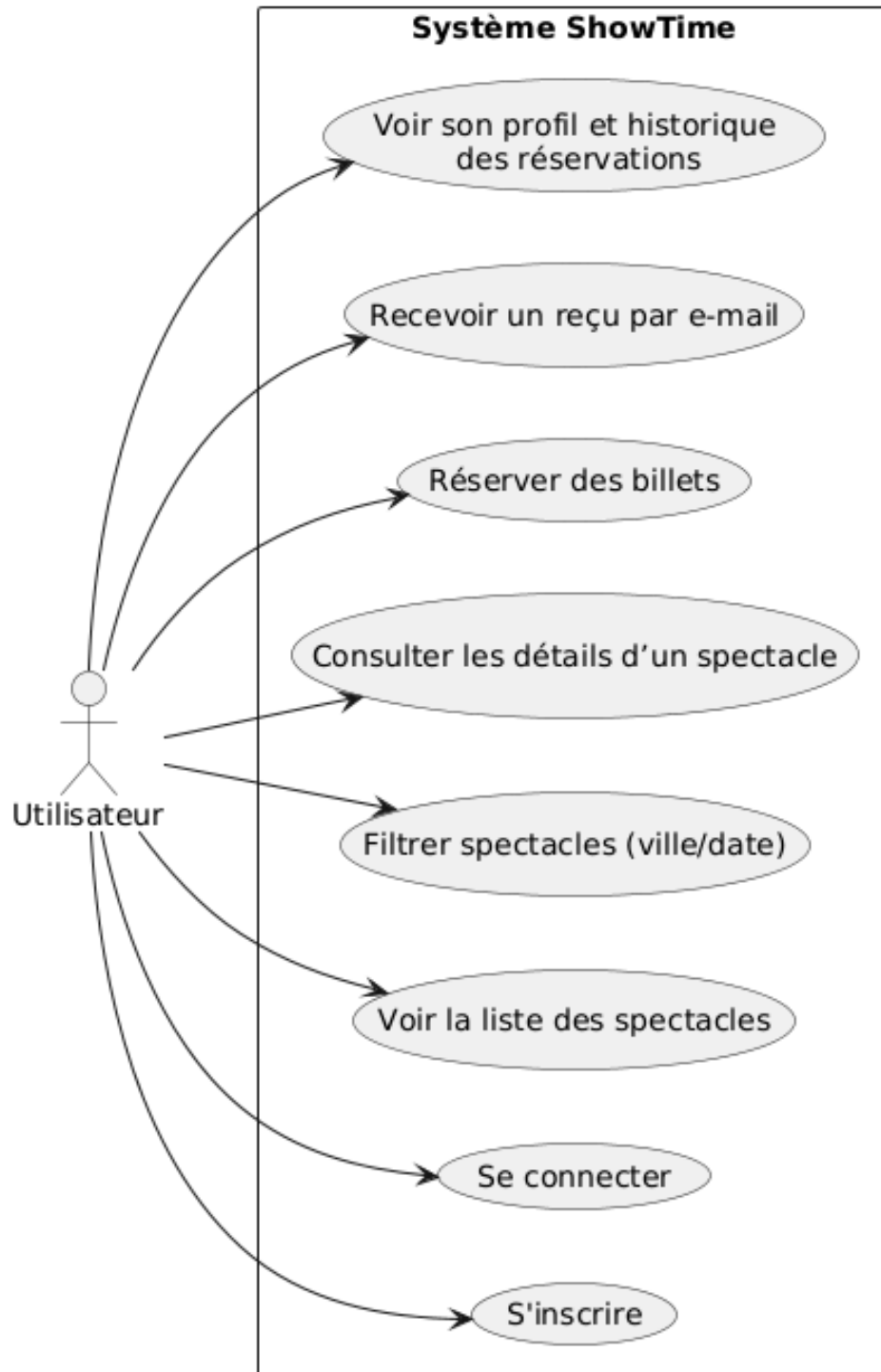
1.3 Technologies utilisées

Voici les principales technologies et outils utilisés dans le cadre du développement de ce projet :

Technologie / Outil	Utilisation
Android Studio / XML	Développement de l'interface utilisateur
Java	Langage principal pour les activités Android
MySQL	Base de données relationnelle
MySQL Workbench	Gestion visuelle de la base de données
Spring Boot	Framework backend pour créer l'API REST
Postman	Test des requêtes API (GET, POST, etc.)
Git / GitHub	Versionning du projet et collaboration

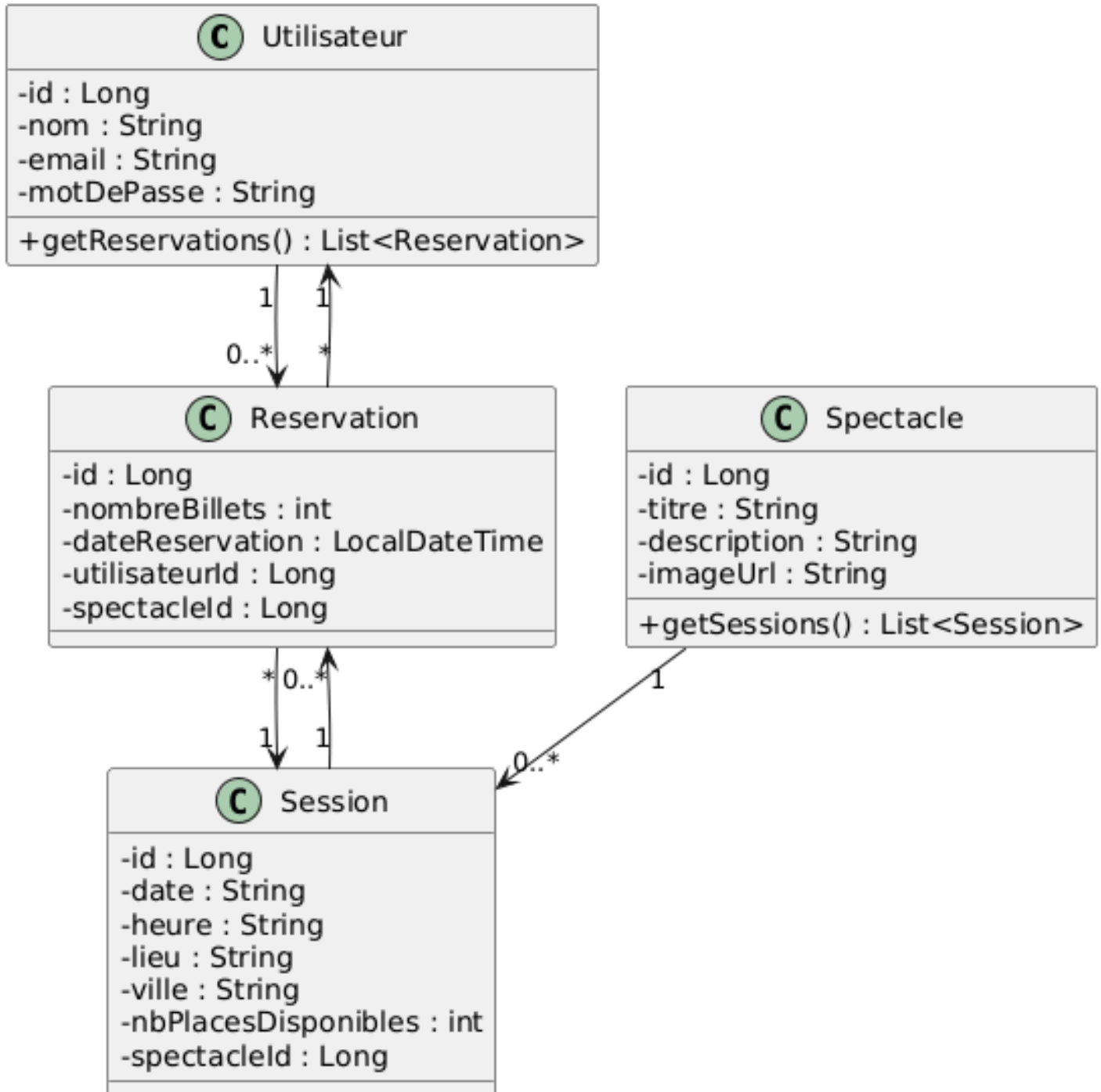
3. Diagrammes UML

3.1 Diagramme de cas d'utilisation



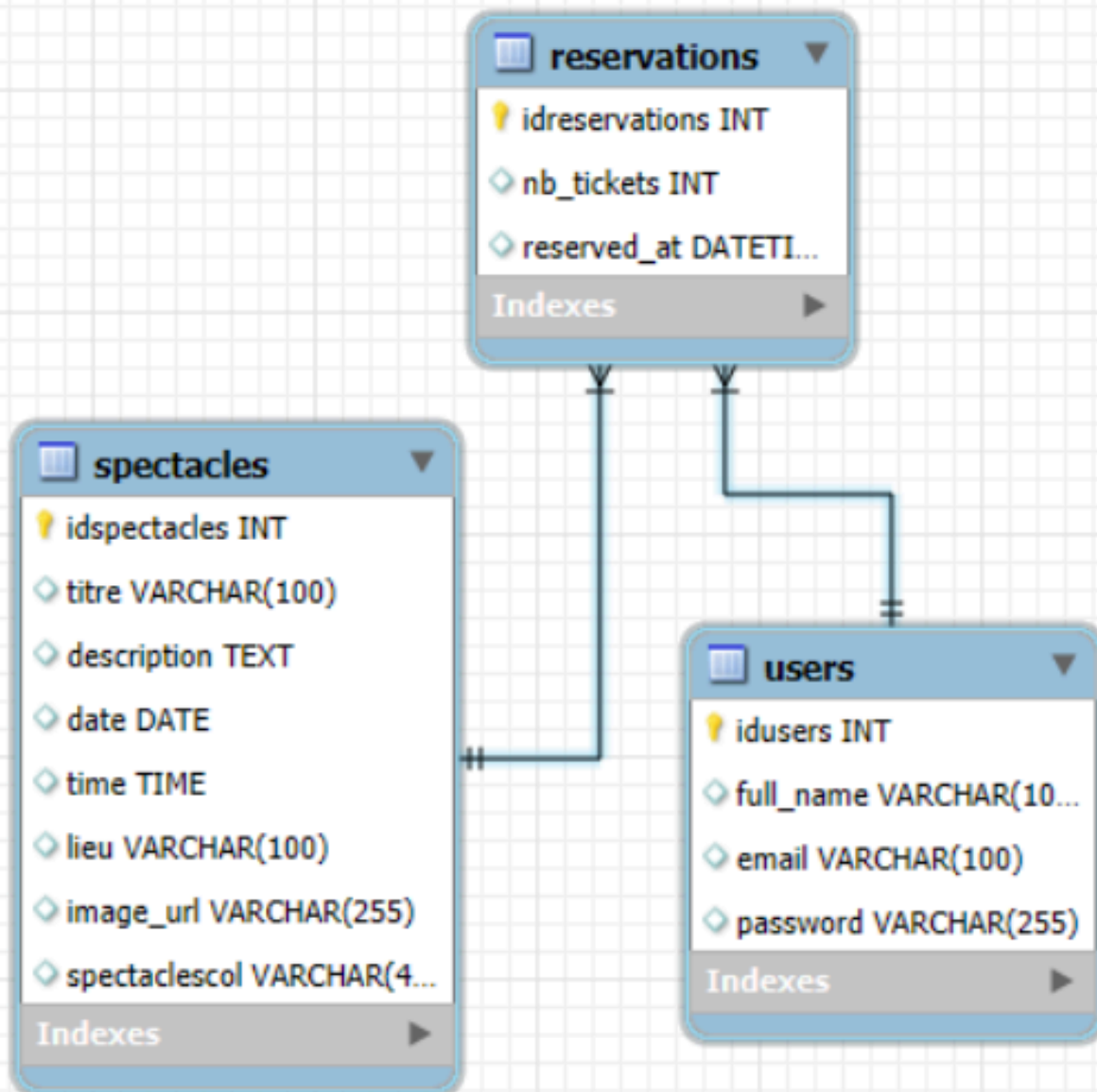
3. Diagrammes UML

3.2 Diagramme de classes



4. Base de Données & API de Connexion

4.1 Modélisation de la base de données



4. Base de Données & API de Connexion

4.2 API Spring Boot : structure et endpoints

L'API backend est développée avec Spring Boot, utilisant Spring Data JPA pour l'accès aux données. Voici l'architecture générale :

Packages :

- **controller** : Contient les *REST controllers* (*UserController, ReservationController...*)
- **service** : Contient la *logique métier*
- **repository** : Interfaces *JPA* pour la communication avec la *BDD*
- **model** : Entités liées aux tables

Principaux endpoints :

Méthode	Endpoint	Description
POST	/api/users/register	Créer un nouveau compte utilisateur
POST	/api/users/login	Connexion avec email & mot de passe
GET	/api/spectacles	Liste des spectacles
GET	/api/spectacles/{id}	Détails d'un spectacle
POST	/api/reservations	Réserver un ou plusieurs billets
GET	/api/reservations/user/{id}	Historique des réservations d'un user

4. Base de Données & API de Connexion

4.3 Sécurité et authentification utilisateur

L'authentification est gérée de manière simple à l'aide de Spring Security avec bcrypt pour le hachage des mots de passe. Lorsqu'un utilisateur se connecte, ses informations sont validées, et un token JWT (JSON Web Token) peut être renvoyé pour sécuriser les appels API (si implémentation future souhaitée).

L'objectif ici est de garantir :

- La confidentialité des mots de passe via chiffrement*
- La séparation entre utilisateurs connectés et non connectés*
- L'accès à certaines fonctionnalités (profil, réservations) seulement si connecté*

4.4 Historique de réservations

Lorsqu'un utilisateur connecté effectue une réservation, celle-ci est enregistrée dans la table reservations avec :

- Le spectacle concerné*
- Le nombre de billets*
- La date de réservation*

Cette information est ensuite consultable via un appel à :

GET /api/reservations/user/{userId}

Cela permet d'alimenter l'écran "Mon Profil" avec un historique complet des réservations, trié par date ou par spectacle.

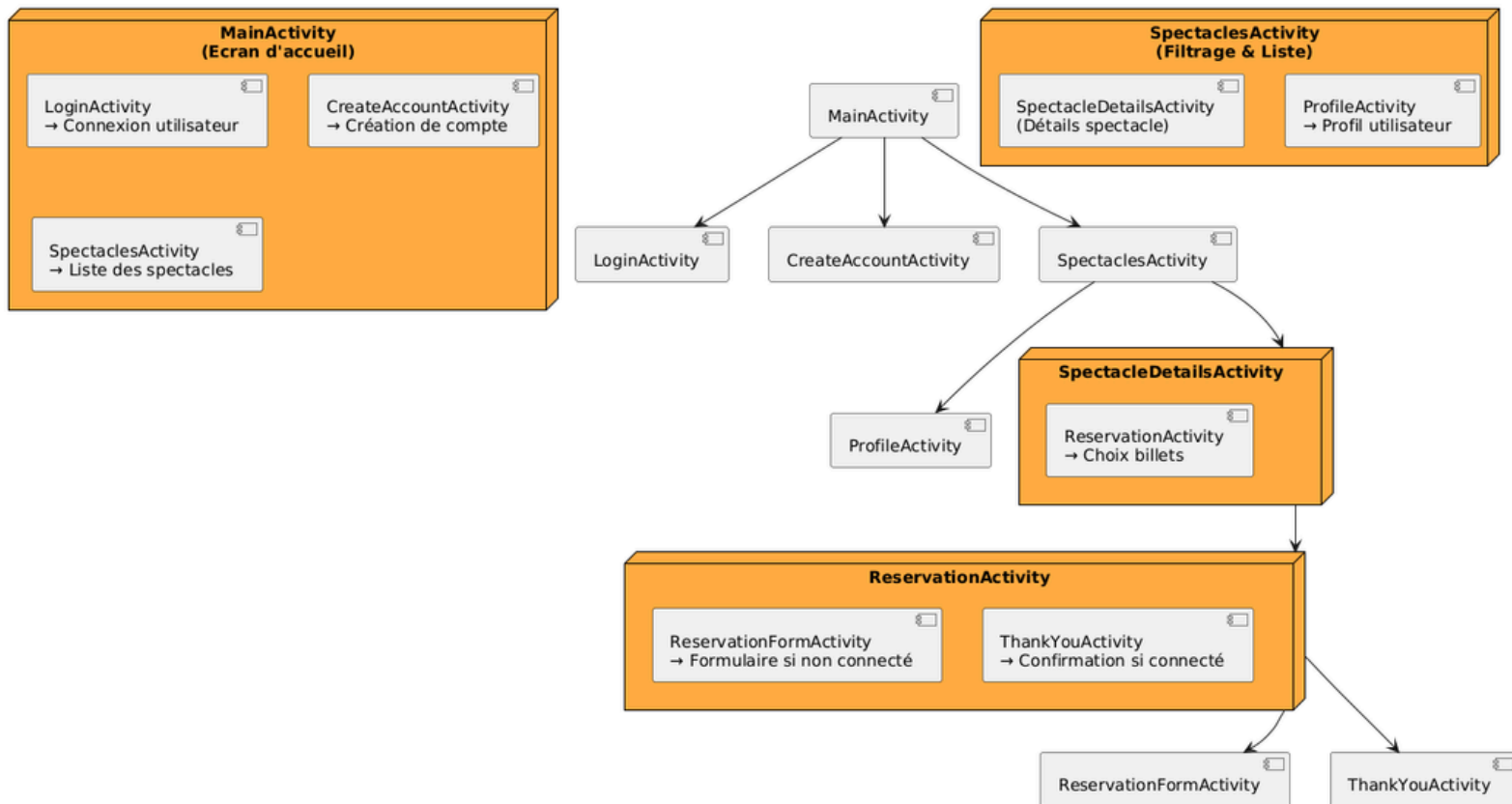
5. Arborescence des Interfaces

5.1 Navigation entre les écrans

- L'utilisateur arrive sur l'écran d'accueil (*MainActivity*).
 - Il peut se connecter (*LoginActivity*)
 - Ou créer un compte (*CreateAccountActivity*)
 - Ou naviguer vers la liste des spectacles (*SpectaclesActivity*)
- Depuis la liste des spectacles
 - Il peut cliquer sur un spectacle pour voir les détails (*SpectacleDetailsActivity*)
 - Depuis les détails, il peut réserver (*ReservationActivity*)
 - Si non connecté → formulaire de réservation (*ReservationFormActivity*)
 - Si connecté → remerciement (*ThankYouActivity*)
- Si connecté, il peut cliquer sur l'icône profil
 - Accès à son profil (*ProfileActivity*)

5. Arborescence des Interfaces

5.2 Arbre des Interfaces (schéma)



6. Outils Utilisés & Difficultés Rencontrées

6.1 Environnement de développement

Le développement de ce projet a été réalisé principalement avec les outils suivants :

- *Android Studio & XML : Pour le développement de l'interface utilisateur mobile native Android.*
- *Spring Tool Suite (STS) : Pour la création de l'API backend avec Spring Boot.*
- *Java : Utilisé à la fois pour le frontend Android (Java Android SDK) et le backend Spring Boot.*
- *MySQL Workbench : Pour la modélisation, la création et la gestion de la base de données relationnelle.*

6.2 Outils tiers utilisés

- *Git & GitHub : Pour la gestion de versions du code source. Cela a permis un suivi structuré de l'évolution du projet.*
- *MySQL Server 8.0 : Système de gestion de base de données relationnelle utilisé pour stocker les utilisateurs et les réservations.*
- *Postman : Utilisé pour tester les requêtes HTTP de l'API Spring Boot (POST, GET, etc.) et vérifier la validité des endpoints.*
- *PlantUML : Pour générer les diagrammes UML tels que les diagrammes de classes, de cas d'utilisation, et l'arborescence des interfaces.*

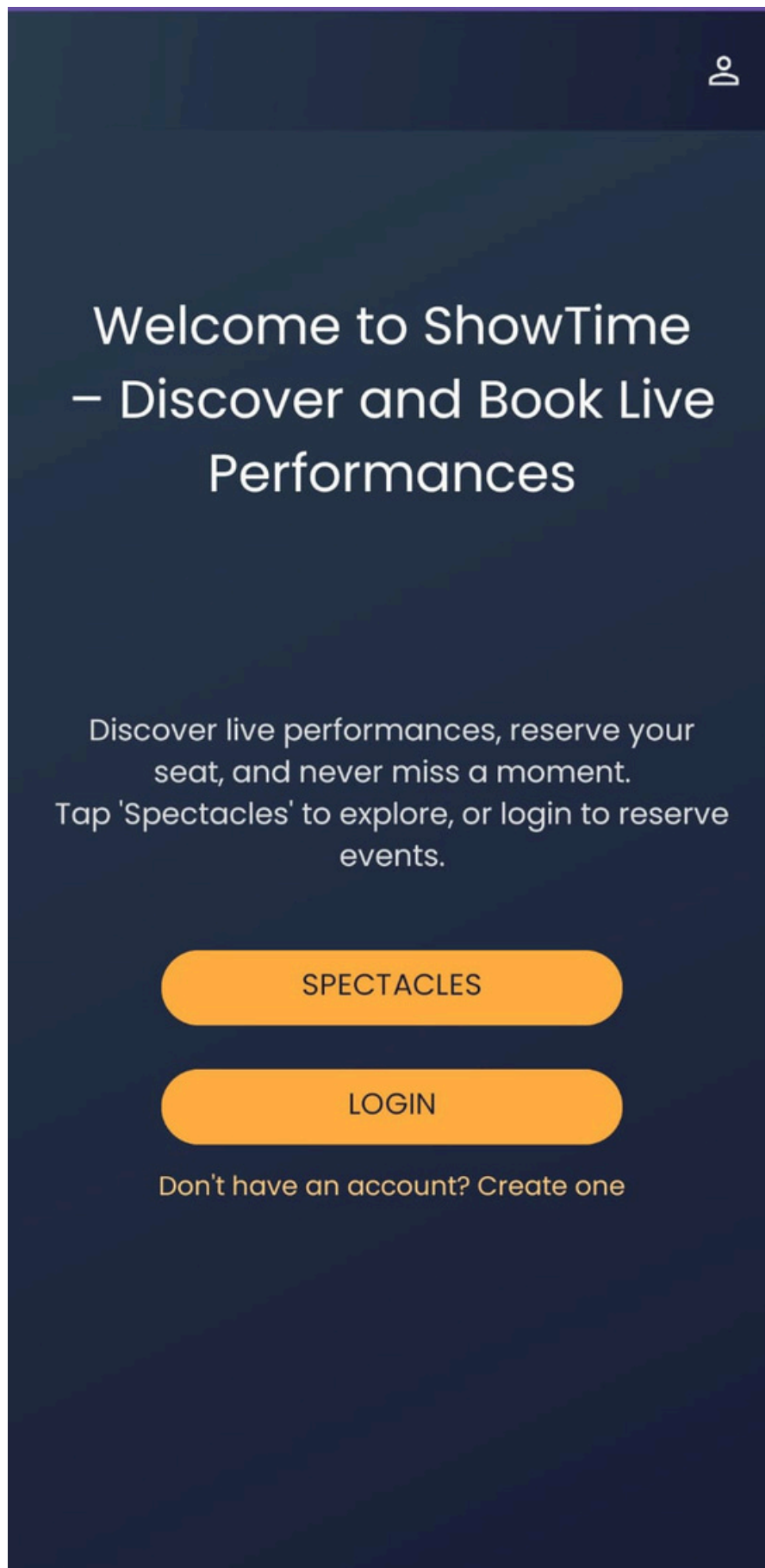
6. Outils Utilisés & Difficultés Rencontrées

6.3 Problèmes rencontrés et solutions apportées

Problème rencontré	Solution apportée
MySQL ne démarrait pas correctement	Réinstallation complète du serveur, configuration manuelle des services et assignation d'un port non utilisé (3306).
Erreur "No database selected" dans Workbench	Sélection manuelle de la base de données en double-cliquant sur son nom dans l'onglet "Schemas".
Connexion entre Spring Boot et MySQL échouée	Mise à jour du application.properties avec l'URL correcte, le driver JDBC et les identifiants de la base.
Design mobile instable (problèmes d'alignement)	Utilisation d'un ScrollView englobant des LinearLayouts et correction des marges/paddings.

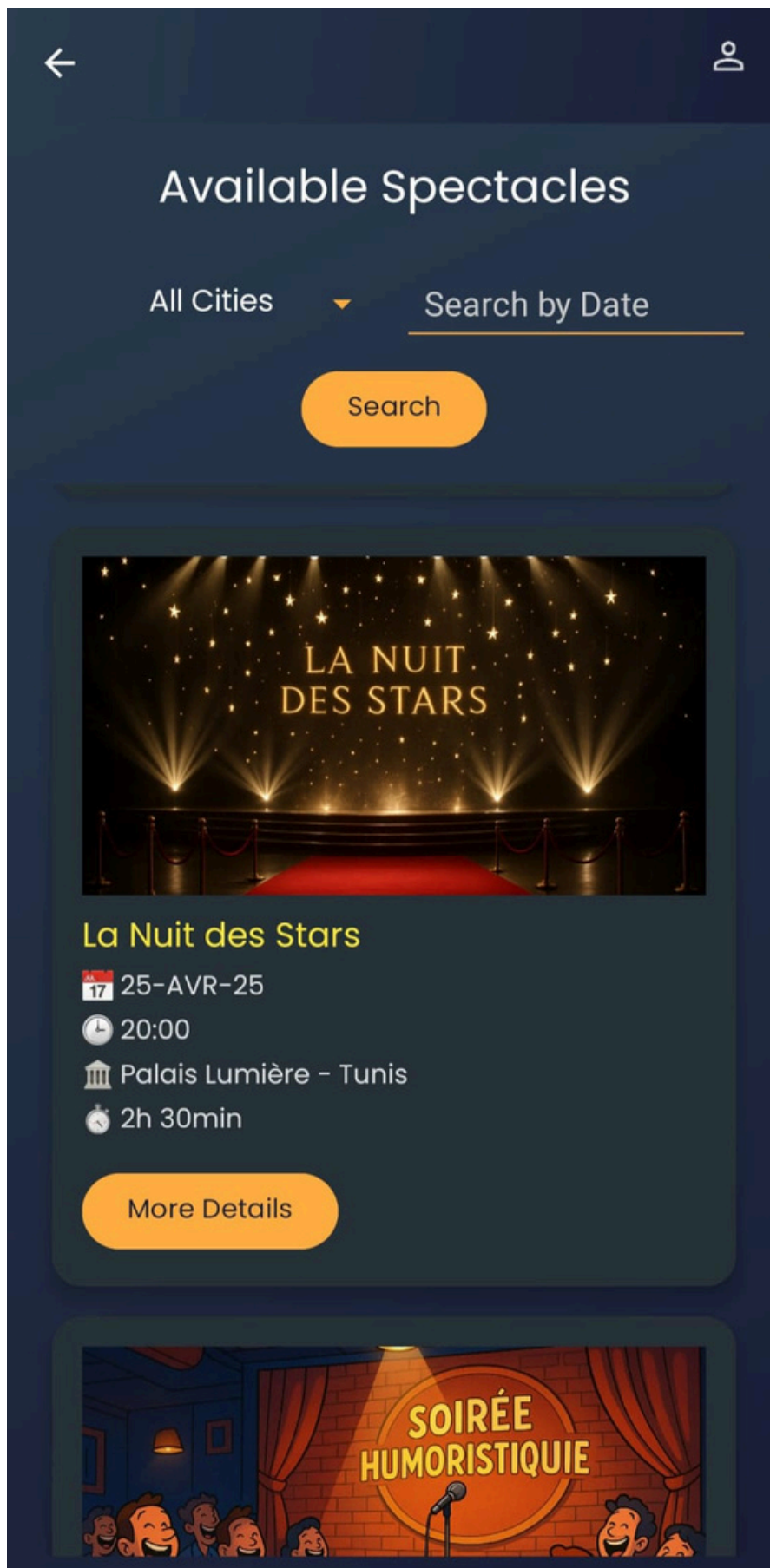
7. Interfaces de l'Application

7.1 Écran d'accueil et login



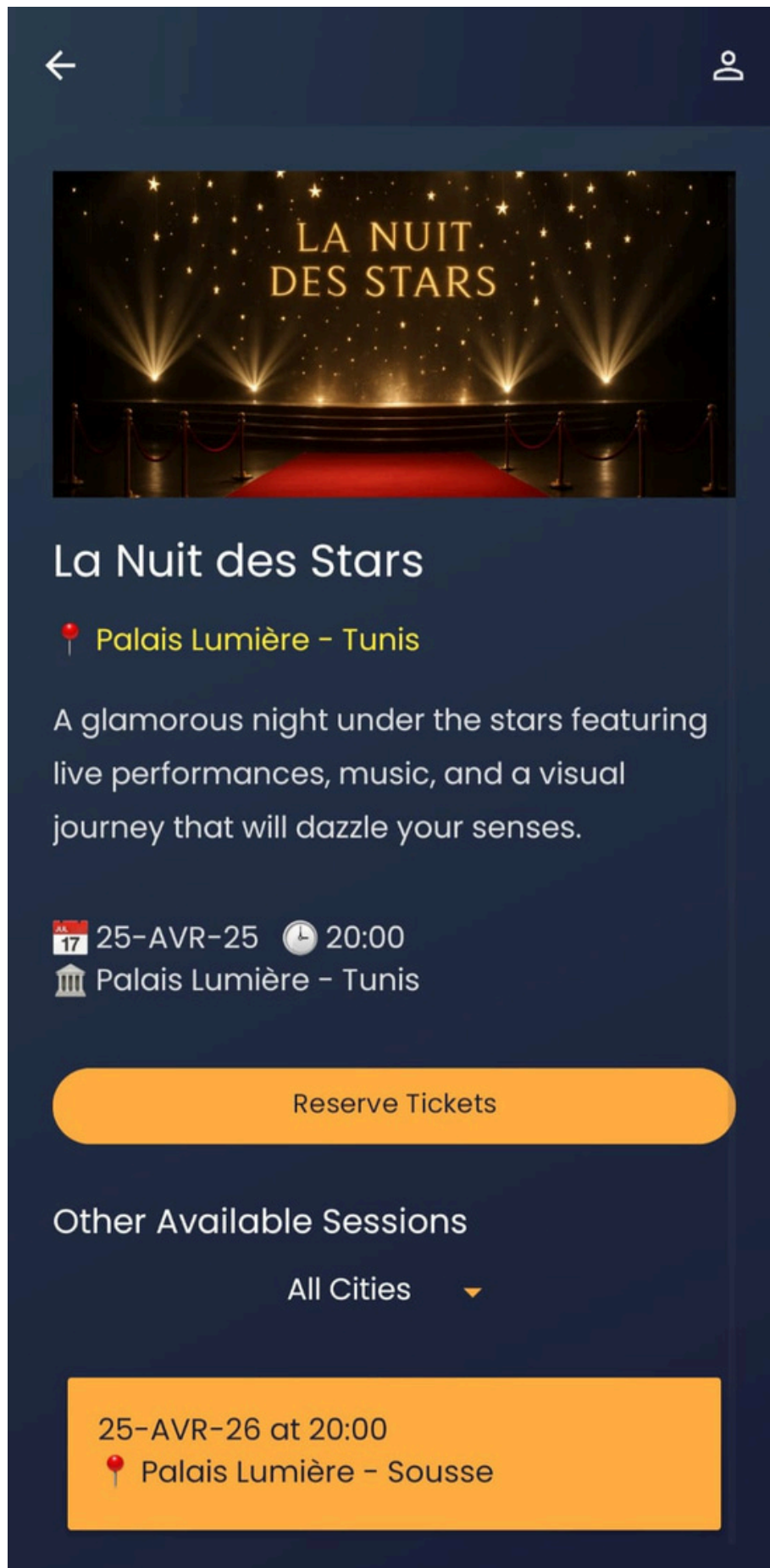
7. Interfaces de l'Application

7.2 Liste des spectacles



7. Interfaces de l'Application

7.3 Détails d'un spectacle



7. Interfaces de l'Application

7.4 Réservation

←

Reserve Tickets





La Nuit des Stars

 20/05/2025

 19:00

 Théâtre de Carthage - Tunis

 Price: 25 DT

 Available Seats: 20

1

▼

 Total: 25 DT

Confirm Reservation

7. Interfaces de l'Application

7.5 Profil utilisateur



8. Annexe : Code des Parties Importantes

8.1 Code backend (Spring Boot)

User.java

```
package tn.enicarthage.model;

import jakarta.persistence.*;
import java.util.List;

@Entity
@Table(name = "users")
public class User {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String fullName;

    @Column(unique = true, nullable = false)
    private String email;

    @Column(nullable = false)
    private String password;

    // ? Lien avec les réservations de l'utilisateur
    @OneToMany(mappedBy = "user", cascade = CascadeType.ALL)
    private List<Reservation> reservations;

    // Constructeurs
    public User() {}

    public User(String fullName, String email, String password) {
        this.fullName = fullName;
        this.email = email;
        this.password = password;
    }

    // Getters et Setters
    public Long getId() { return id; }
    public String getFullName() { return fullName; }
    public void setFullName(String fullName) { this.fullName = fullName; }
    public String getEmail() { return email; }
    public void setEmail(String email) { this.email = email; }
    public String getPassword() { return password; }
    public void setPassword(String password) { this.password = password; }
    public List<Reservation> getReservations() { return reservations; }
    public void setReservations(List<Reservation> reservations) { this.reservations = reservations; }
```

8. Annexe : Code des Parties Importantes

8.1 Code backend (Spring Boot)

UserRepository.java

```
package tn.enicarthage.repository;

import tn.enicarthage.model.User;
import org.springframework.data.jpa.repository.JpaRepository;
import java.util.Optional;

public interface UserRepository extends JpaRepository<User, Long> {
    Optional<User> findByEmail(String email);
}
```

UserService.java

```
package tn.enicarthage.service;

import tn.enicarthage.model.User;
import tn.enicarthage.repository.UserRepository;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;

import java.util.Optional;

@Service
public class UserService {

    @Autowired
    private UserRepository userRepository;

    public User register(User user) {
        return userRepository.save(user);
    }

    public Optional<User> login(String email, String password) {
        return userRepository.findByEmail(email)
            .filter(u -> u.getPassword().equals(password));
    }

    public Optional<User> getById(Long id) {
        return userRepository.findById(id);
    }
}
```


8. *Annexe : Code des Parties Importantes*

8.1 *Code backend (Spring Boot)*

ReservationService.java

```
public List<Reservation> getReservationsForUser(Long userId) {  
    return reservationRepository.findByUserId(userId);  
}  
  
public Reservation createReservation(Long userId, Long spectacleId, int nbTickets) {  
    Reservation reservation = new Reservation();  
    reservation.setUser(userRepository.findById(userId).orElseThrow());  
    reservation.setSpectacle(spectacleRepository.findById(spectacleId).orElseThrow());  
    reservation.setNbTickets(nbTickets);  
    reservation.setReservedAt(LocalDateTime.now());  
    return reservationRepository.save(reservation);  
}
```

8. Annexe : Code des Parties Importantes

8.2 Code frontend Android (activités & adaptateurs)

MainActivity.java

Button btnSpectacles;

@Override

```
protected void onCreate(Bundle savedInstanceState) {  
    super.onCreate(savedInstanceState);  
    setContentView(R.layout.activity_main);
```

```
    ImageView profileIcon = findViewById(R.id.profileIcon);
```

```
    profileIcon.setOnClickListener(v -> {
```

```
        SharedPreferences prefs = getSharedPreferences("UserPrefs",  
MODE_PRIVATE);
```

```
        String email = prefs.getString("email", "");
```

```
        if (email.isEmpty()) {
```

```
            // Not logged in → Go to Login Page
```

```
            startActivity(new Intent(this, LoginActivity.class));
```

```
        } else {
```

```
            // Logged in → Go to Profile Page
```

```
            startActivity(new Intent(this, ProfileActivity.class));
```

```
        }
```

```
    });
```

```
    btnSpectacles = findViewById(R.id.btnSpectacles);
```

```
    Button btnLogin = findViewById(R.id.btnLogin);
```

```
    TextView btnCreateAccount = findViewById(R.id.btnCreateAccount);
```

```
    Button btnSpectacles = findViewById(R.id.btnSpectacles);
```

```
    btnSpectacles.setOnClickListener(v -> {
```

```
        Intent intent = new Intent(MainActivity.this, SpectaclesActivity.class);
```

```
        startActivity(intent);
```

```
    });
```

```
    btnLogin.setOnClickListener(v -> {
```

```
        Intent intent = new Intent(MainActivity.this, LoginActivity.class); // rename  
AdminLoginActivity to LoginActivity
```

```
        startActivity(intent);
```

```
    });
```

```
    btnCreateAccount.setOnClickListener(v -> {
```

```
        Intent intent = new Intent(MainActivity.this, CreateAccountActivity.class);
```

```
        startActivity(intent);
```

```
    });
```

```
}
```

```
}
```


8. Annexe : Code des Parties Importantes

8.2 Code frontend Android (activités & adaptateurs)

LoginActivity.java

```
public class LoginActivity extends AppCompatActivity {

    private EditText editEmail, editPassword;
    private Button btnLogin;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_login);

        Toolbar toolbar = findViewById(R.id.toolbar);
        setSupportActionBar(toolbar);
        getSupportActionBar().setDisplayHomeAsUpEnabled(true);
        getSupportActionBar().setTitle("Login");

        toolbar.setNavigationOnClickListener(v -> finish());

        editEmail = findViewById(R.id.editEmail);
        editPassword = findViewById(R.id.editPassword);
        btnLogin = findViewById(R.id.btnLogin);

        btnLogin.setOnClickListener(v -> {
            String email = editEmail.getText().toString().trim();
            String password = editPassword.getText().toString().trim();

            if (isValidLogin(email, password)) {
                Toast.makeText(this, "Login successful ✅", Toast.LENGTH_SHORT).show();
                finish(); // Just close LoginActivity and return to MainActivity or Spectacles
            } else {
                Toast.makeText(this, "Invalid credentials ❌", Toast.LENGTH_SHORT).show();
            }
        });
    }

    private boolean isValidLogin(String email, String password) {
        // Later we will check from database
        // For now you can allow anything not empty
        return !email.isEmpty() && !password.isEmpty();
    }
}
```

9. Conclusion Générale

Au terme de ce projet de développement d'une application mobile de réservation de spectacles, nous avons pu explorer l'ensemble du cycle de création d'une solution mobile complète, de la conception des interfaces utilisateur jusqu'à l'implémentation d'une base de données et d'un backend sécurisé avec Spring Boot et MySQL.

Ce travail nous a permis de mettre en pratique de nombreuses compétences en développement Android, gestion de la navigation entre les écrans, intégration d'authentification, et communication client-serveur via API. L'interface a été pensée pour être fluide, intuitive, et centrée sur l'expérience utilisateur.

Les différentes fonctionnalités proposées — telles que la réservation de billets, l'affichage des détails par spectacle, la gestion de sessions multiples et l'espace personnel utilisateur — répondent aux attentes d'un système moderne de billetterie en ligne.

Ce projet représente ainsi une base solide, évolutive et facilement extensible, qui pourrait à terme être connectée à une infrastructure réelle et déployée à large échelle.